CMCMIS_SIMPLIFIED

PHASE 7 SLICE 2 — SEALED

Job Request Detail + Approve/Reject + Auto Job Card

End-to-End Technical Documentation · Senior Engineer Build Record

Field	Value
Document	phase7SLICE2twofullySEALEDsealpacked.docx
Version	v1.0 · SEALED · DS-approved build record
Project	CMCMIS_SIMPLIFIED — Computerized Maintenance & Calibration Management Information System
Phase / Slice	Phase 7 · Slice 2
Scope	Job Request Detail, Convert/Approve+Assign, Reject, auto-create Job Card, Conversion workspace
Code posture	JavaScript only · .js / .jsx · no TypeScript
Database posture	MySQL final · additive-only migrations · legacy table preservation
Primary roles	Normal User, View-Only, Lab Engineer, Lab In-Charge, Super Admin
Prepared for	Deep Sorathiya (DS)
Generated	2026-05-19

SEALED STATEMENT This document treats the pasted Phase 7 Slice 2 implementation transcript as the build record. It explains what was built, why it was built this way, how the database/backend/frontend logic was developed, and how future engineers should preserve the slice without rediscovering the reasoning.

Table of Contents

READING RULE Read Part IV before touching SQL, Part VI before touching any status mutation, Part VIII before modifying service/repo boundaries, and Part XIV before relaxing any permission gate.

Part I — Executive Seal & Mission Briefing

MISSION Phase 7 Slice 2 converts the job-flow foundation from a submitted-request registry into a controlled workflow engine: Lab In-Charge/Super Admin can open a Job Request detail page, convert it into an assigned Job Card atomically, or reject it with a mandatory reason.

This slice closes the exact gap intentionally left by the previous Job Request + Job Card foundation: the system could create and list requests, but it could not yet perform the managerial decision that turns a request into executable work. The new slice adds the decision layer, the conversion transaction, the rejection terminal path, and the UI surfaces needed by Lab In-Charge users to process the queue with confidence.

The implementation is not a redesign of Phase 6. It is an additive continuation. The existing legacy tables stay authoritative. The new code follows the sealed layered-module pattern: validators → repository → service → controller → routes on the backend, and api → hooks → page orchestrator → reusable components on the frontend.

Outcome	Delivered meaning
Job Request Detail	A complete read-only view with equipment, submitter, complaint, accessories, timeline, and linked Job Card summary.
Convert / Approve + Assign	One user action creates a Job Card and updates the Job Request in one database transaction.
Reject	A terminal SUBMITTED → REJECTED path with mandatory reason, audit, and history.
Auto Job Card	New Job Card row is inserted from JR snapshots with engineer, workflow, timeline, resources, and parent-JR linkage.
Conversion Workspace	A manager-facing queue with tabs, filters, conversion modal, reject modal, and cache invalidation.

SENIOR ENGINEER SUMMARY The core engineering win is not the UI. The win is the atomic conversion boundary: no partial approval, no orphan Job Card, no missing status history, no audit without a state change, and no state change without audit intent.

What makes this slice difficult?

- The legacy database already contains more than nineteen thousand Job Cards and more than twenty-one thousand Job Requests, so destructive migrations are forbidden.
- The intuitive state APPROVED does not exist in JR_MVP_STATUS enum, forcing a senior-engineering decision: treat APPROVED as a transient logical state captured in status history, not as a persisted row state.
- The Job Card identifier has a legacy varchar(9) PK format, so the new generator must fit the same column without breaking old values.
- The conversion transaction touches multiple entities: Job Request, Job Card, status history, audit log, KPI cache invalidation, and frontend cache refresh.
- Normal users must retain row-level visibility restrictions while LIC/SA users need global conversion capability.

Part II — Source Material and Build Evidence

The documentation is generated from the two pasted implementation transcripts uploaded for this request. The first captured preparation, schema discovery, decisions, endpoint map, audit vocabulary, and smoke matrix. The second captured the green-flag build sequence: migrations, backend modules, frontend hooks, routes, pages, modals, and verification checkpoints.

Source artifact	Role in this document
Pasted text.txt	Prep report, SCHEMA_PHASE7_SLICE2.md decisions, Step-0 introspection, endpoint map, audit vocabulary, smoke matrix, and DS green-flag context.
Pasted text (2).txt	Implementation transcript covering migrations 200/201, backend state machine/repo/service/controller/routes, frontend schemas/hooks/pages/modals, and conversion workspace code.
phase6FINALsealedLOCKED.docx	Previous sealed doctrine: Phase 6 Slice 1 deliberately deferred approve/reject and auto Job Card creation to later slices.
FINAL-DESC-CMCMIS.pdf	Project-level locked plan: Job Requests include approve/reject/state machine; Job Cards include auto-create on approval.

EVIDENCE RULE This document does not dump the source code. It documents the technical intent, decisions, data flow, transaction boundaries, and operational rules behind the code, with only light pseudo-code where it clarifies logic.

Build transcript milestones extracted

Milestone	Evidence captured
Prep / discovery	Explored jobRequests, jobCards, middleware, lookups, DB introspection, and wrote SCHEMA_PHASE7_SLICE2.md.
DB migrations	Migration 200 added five NULLable Job Card columns; migration 201 added workload lookup indexes.
Backend implementation	State machine, repositories, services, controllers, routes, validators, and engineers lookup were extended.
Frontend implementation	Schemas, API wrappers, hooks, App routes, detail page components, conversion page, and modals were added.
Verification hooks	BE module load check succeeded; DB column/index existence was verified; smoke matrix A1-A14 was defined.

Part III — Phase Context: Why Slice 2 Exists

Phase 6 Slice 1 delivered the durable foundation: Job Request list, Job Request creation/submit flow, Job Card list, state-machine baseline, audit pairing doctrine, and row-level scope. It intentionally did not deliver the detail page, approve/reject flows, or auto Job Card creation hook. Phase 7 Slice 2 is the focused vertical slice that implements those missing workflow verbs.

VERTICAL SLICE LOGIC A slice must cross database, backend, frontend, RBAC, audit, and verification. Phase 7 Slice 2 is therefore not “just a page” and not “just an endpoint”; it is a controlled business transition through the full system.

Before and after

Before Slice 2	After Slice 2
JR could be created and submitted.	JR can be opened in full detail and actioned by authorized roles.
Job Cards could be listed.	Job Cards can be auto-created from submitted Job Requests.
Status machine supported submit.	Status machine supports submit, approve, assign, and reject logic.
Approval was only a future doctrine.	Conversion is an executable transaction with history and audit.
No conversion queue.	Conversion workspace groups pending work by Calibration, Inspection, and Master Data Correction.

Role-level meaning

Role	Meaning in Slice 2
Normal User	Can open own Job Request details; cannot convert or reject.
View-Only	Can inspect detail where permissions allow; cannot mutate state.
Lab Engineer	Can see system work context but does not own approval authority in this slice.
Lab In-Charge	Primary operator for conversion and rejection.
Super Admin	Has full administrative conversion/rejection authority.

Part IV — Ground Truth Schema Introspection

DATABASE POSTURE The database was not designed as a greenfield schema. It is a legacy-production-compatible schema. Therefore the implementation begins with introspection, not imagination.

Fact	Value	Engineering implication
cmms_jobrequest_mst row count	21,491	Legacy data dominates. New columns must be NULLable; no destructive backfills.
cmms_jobcard_mst row count	19,432	Job Card additions must not break existing rows.
JR_MVP_STATUS enum	DRAFT, SUBMITTED, ASSIGNED, IN_PROGRESS, COMPLETED, VERIFIED_CLOSED, REJECTED, REOPENED	APPROVED is not persisted as JR status.
Approval/rejection columns	JR_APPROVED_BY/ON, JR_REJECTED_BY/ON, JR_REJECTION_REASON already exist	No ALTER needed on Job Request master for these fields.
Assignment column	JR_ASSIGNED_ENGINEER varchar(7) already exists	Reuse existing employee_id shape.
Linked JC field	JR_SECTIONJOB_NO varchar(9) FK to JM_SectionJobNo	Acts as linked Job Card ID; no new linked_job_card_id column.
Status history table	to_status varchar(30)	Can store APPROVED as text without changing JR enum.
Engineer lookup	1 active LAB_ENGINEER user in introspection	Functional for smoke; more engineers can be seeded later.

CRITICAL DISCOVERY The prompt could have led to new columns on cmms_jobrequest_mst, but introspection proved the required approval/rejection/assignment/link fields already existed. Senior engineering avoided unnecessary schema expansion.

Job Card column additions chosen

New column	Type	Why it exists
JM_ASSIGNED_ENGINEER	varchar(7) NULL	Stores the engineer employee_id assigned at conversion time.
JM_WORKFLOW_TYPE	varchar(50) NULL	Stores scoped workflow flavor such as CALIBRATION_STANDARD or INSPECTION_DETAILED.
JM_REQUIRED_RESOURCES	varchar(2000) NULL	Carries tools/resources instructions into the Job Card.
JM_SPECIAL_INSTRUCTIONS	varchar(2000) NULL	Carries special safety/procedure notes to the engineer.
JM_PARENT_JR_NO	int(11) NULL	Inverse linkage from Job Card back to parent Job Request.

Index additions chosen

Index	Columns	Purpose
idx_jc_engineer_status	JM_ASSIGNED_ENGINEER, JM_MVP_STATUS	Fast engineer workload counts for dropdown.
idx_jc_parent_jr	JM_PARENT_JR_NO	Fast reverse lookup from parent JR to Job

		Card.

Part V — Decision Register D-7.2.1 → D-7.2.12

ID	Topic	Locked decision
D-7.2.1	Convert atomicity	One button triggers one transaction: approve + assign + Job Card insert + history + audit. Any failure rolls back all.
D-7.2.2	Reject flow	Separate terminal SUBMITTED → REJECTED transition with mandatory reason. It never creates a Job Card.
D-7.2.3	APPROVED transient	APPROVED does not persist in JR_MVP_STATUS. It is captured only in status_history because the DB enum has no APPROVED value.
D-7.2.4	Two history rows per Convert	The timeline shows SUBMITTED → APPROVED and APPROVED → ASSIGNED, even though the JR row ends at ASSIGNED.
D-7.2.5	Job Card ID generator	New MVP format J + zeroPad(JM_JobCardNO, 8), fitting varchar(9) and avoiding legacy collisions.
D-7.2.6	Convert DB state	Final JR_MVP_STATUS is ASSIGNED; approval and assigned engineer fields update atomically.
D-7.2.7	JC ALTERs	Add only five NULLable columns to Job Card master. No destructive migrations.
D-7.2.8	Engineer dropdown source	All active LAB_ENGINEER users system-wide, sorted by active_card_count.
D-7.2.9	Engineer identity	Store employee_id varchar(7), not numeric user_id, to match the rest of the schema.
D-7.2.10	Workflow type	Six scoped workflow strings enforced in validators and UI, not with DB CHECK constraints.
D-7.2.11	Post-convert UX	Close modal, stay on /conversion, refetch tab lists and badges, show success message.
D-7.2.12	Save as Draft	Visible but disabled in Slice 2; partial conversion save is deferred.

DS GREEN FLAG The transcript records DS approval to go with the recommended answers. That means the decision register is sealed for this slice unless future requirements explicitly reopen it.

Why this register matters

The decision register is more important than the code diff. Code can be rewritten; the register preserves the why behind every edge-case choice. The next engineer should not ask again why APPROVED is not stored in JR_MVP_STATUS or why employee_id is stored instead of user_id. Those answers are frozen here.

Part VI — State Machine Doctrine

STATE MACHINE RULE No module may update JR_MVP_STATUS directly without passing through the state-machine transition function. The route and service may ask for a transition; the state machine decides if it is legal.

```
DRAFT       --submit-->  SUBMITTED
SUBMITTED   --approve--> APPROVED   (logical only, not persisted in JR enum)
APPROVED    --assign-->  ASSIGNED   (final persisted state after Convert)
SUBMITTED   --reject-->  REJECTED   (terminal for Slice 2)
```

Persisted vs logical states

State	Persisted in JR_MVP_STATUS?	Where visible	Reason
DRAFT	Yes	JR row and list/detail UI	Creation baseline.
SUBMITTED	Yes	JR row, conversion queue, dashboard pending count	Ready for LIC/SA action.
APPROVED	No	Status history only	Avoids destructive enum modification.
ASSIGNED	Yes	JR row and linked Job Card	Final result of successful conversion.
REJECTED	Yes	JR row and timeline	Terminal in Slice 2.

Transition permission matrix

From	Action	To	Permission	Owner required?	Slice
DRAFT	submit	SUBMITTED	job_request:create	Yes	Phase 6 Slice 1
SUBMITTED	approve	APPROVED	job_request:approve	No	Phase 7 Slice 2
APPROVED	assign	ASSIGNED	job_request:assign-engineer	No	Phase 7 Slice 2
SUBMITTED	reject	REJECTED	job_request:reject	No	Phase 7 Slice 2

WHY TWO PERMISSIONS FOR CONVERT? Convert chains approve and assign. The route checks both job_request:approve and job_request:assign-engineer, and the state machine checks them again. This guards against future route wiring mistakes.

Part VII — Database Development Logic

The database logic follows three rules: preserve legacy rows, add only nullable structures, and use explicit indexes for new query patterns. Slice 2 changes Job Cards because the new Auto Job Card carries data that legacy rows did not previously store.

Migration 200 — Why each column exists

Column	Database role	Application role
JM_ASSIGNED_ENGINEER	New nullable employee_id storage	Lets Job Card list/detail show who owns the work.
JM_WORKFLOW_TYPE	New nullable workflow string	Lets the engineer know which workflow path to follow.
JM_REQUIRED_RESOURCES	Optional instruction field	Carries tools, fixtures, devices, or reference standards.
JM_SPECIAL_INSTRUCTIONS	Optional instruction field	Carries safety notes, constraints, handling instructions.
JM_PARENT_JR_NO	Reverse parent link	Allows direct lookup of the parent Job Request from Job Card.

Migration 201 — Why the indexes exist

The new engineer dropdown is not a static user list. It is a workload-aware selector. For each engineer, the system calculates active assigned cards. Without an engineer/status composite index, that dropdown would scan many Job Card rows repeatedly. The new index converts that workload count into a narrow seek.

Query pattern	Index response
Count active Job Cards for one engineer	idx_jc_engineer_status supports engineer + status filter.
Find Job Card created from one Job Request	idx_jc_parent_jr supports reverse lookup.

MIGRATION DISCIPLINE Even though new fields are semantically required for new rows, they remain NULLable in the database because legacy rows already exist. New-row completeness is enforced in validators and service logic, not by retroactively forcing old data to satisfy new rules.

Job Card ID generation

```
new JM_JobCardNO = SELECT MAX(JM_JobCardNO) + 1 FOR UPDATE
new JM_SectionJobNo = "J" + zeroPad(JM_JobCardNO, 8)
example: JM_JobCardNO = 24214  =>  JM_SectionJobNo = J00024214
```

The generator fits the existing varchar(9) primary key and avoids collisions with legacy section-job numbers, which use numeric-looking formats such as 62026043.

Part VIII — Backend Development Logic

BACKEND PATTERN The backend keeps the sealed module pattern: validators define accepted input, repositories know SQL, services own transactions and business orchestration, controllers remain thin, routes own middleware ordering.

Backend file groups touched

Layer	Files / areas	Responsibility
State machine	jobRequests.stateMachine.js	Adds approve, assign, reject transitions and illegal-transition errors.
Job Requests repo	jobRequests.repo.js	Detail query, history query, convert update, reject update.
Job Cards repo	jobCards.repo.js	nextSectionJobNo and insertFromJobRequest.
Lookups repo/controller/routes	lookups.*	Engineers dropdown with active_card_count.
Validators	jobRequests.validators.js	convertSchema and rejectSchema.
Service	jobRequests.service.js	getDetail, getHistory, convertToJobCard, reject orchestration.
Controller	jobRequests.controller.js	HTTP envelope and next(error) handling.
Routes	jobRequests.routes.js	GET detail/history, POST convert/reject with middleware gates.

Repository thinking

The repository layer follows the legacy-to-canonical aliasing doctrine. SQL returns canonical shapes to services and frontend-facing serializers, while preserving real table and column names internally. This prevents legacy schema names from leaking across every layer.

Repository function	Why it exists
findByIdWithDetails	Returns the full Job Request detail payload with submitter, division, accessories, linked Job Card, and names.
findHistory	Returns ordered status history rows for the Timeline component.
updateOnConvert	Updates JR into ASSIGNED with approval fields, engineer, linked JC, and timestamp.
updateOnReject	Updates JR into REJECTED with rejected-by, rejected-on, and reason.
insertFromJobRequest	Creates the Job Card from JR snapshot + modal payload inside the same transaction.
listEngineers	Returns active LAB_ENGINEER users with workload counts.

Service orchestration thinking

The service layer is the transaction boundary. It knows the business process, not just the SQL. It locks the Job Request row, checks the state machine, generates the Job Card sequence, inserts the Job Card, updates the Job Request, appends history rows, writes audit rows, commits, and then invalidates caches.

```
convertToJobCard(jrNo, body, actor):
  begin transaction
  lock JR row
  require status == SUBMITTED
  transition(SUBMITTED, approve, actor)
```

```
append history SUBMITTED -> APPROVED
transition(APPROVED, assign, actor)
generate Job Card number
insert Job Card from JR snapshot + modal fields
update JR to ASSIGNED + approval + engineer + linked JC
append history APPROVED -> ASSIGNED
write JR_CONVERT and JC_CREATE audit rows
commit
invalidate KPI and list caches
```

ATOMICITY GUARANTEE The service must never commit a Job Card without updating the parent Job Request, and must never update the parent Job Request without creating the Job Card. The transaction is the product.

Part IX — Frontend Development Logic

FRONTEND PATTERN The frontend uses the same discipline as previous phases: API wrappers talk to backend endpoints, hooks own fetch/cache/invalidation, pages orchestrate, and components render small pieces.

Frontend area	What was added
Schemas	jobRequestConvertSchema, jobRequestRejectSchema, workflow buckets and labels.
API wrappers	fetch detail, fetch history, convert, reject, engineers lookup.
Hooks	useJobRequestDetail, useJobRequestHistory, useEngineersLookup, useConversionList.
Routing	New /job-requests/:id and /conversion routes.
Detail page	Header, equipment, submission, complaint, timeline, linked Job Card, action bar.
Conversion page	Three tabs, filter strip, table, Convert modal, Reject modal.

Schema thinking

The frontend schema mirrors the backend schema to catch obvious form errors before network submission. It does not replace backend validation. The backend remains authoritative, especially for permissions, workflow type mismatch, and illegal state transitions.

Schema	Key rule
convertSchema	Engineer required, workflow type required, received/planned/target dates ordered, optional resources/instructions bounded.
rejectSchema	Reason required with minimum length and DB-safe maximum length.
workflow buckets	Calibration requests only show calibration workflows; repair shows inspection workflows; registration shows master data workflows.

Hook thinking

The hooks are deliberately cache-aware. A successful convert or reject invalidates detail cache, history cache, conversion-list cache, engineers lookup cache, and the Job Request list cache. This makes the UI feel current without forcing a full page reload.

Hook	Responsibility
useJobRequestDetail	Fetches one detail payload and exposes refetch/invalidate.
useJobRequestHistory	Fetches append-only timeline rows.
useEngineersLookup	Fetches active engineers and workload counts.
useConversionList	Fetches SUBMITTED requests per tab with polling.

Part X — Convert Workflow Deep Dive

Convert is the most important flow in this slice. It is not simply “approve”; it approves and assigns in the same business action, creating a Job Card that the engineering team can execute.

Convert UI sections

Section	Inputs	Purpose
Assignment & Workflow	Engineer dropdown, workflow type dropdown	Chooses who owns the work and what execution workflow applies.
Timeline	Equipment received date, planned start, target end	Creates a planning frame before the engineer starts work.
Instructions	Required resources, special instructions	Preserves manager instructions inside the Job Card.

Convert transaction write set

```
BEGIN
  SELECT JR FOR UPDATE
  INSERT history: SUBMITTED -> APPROVED
  INSERT cmms_jobcard_mst: new ASSIGNED Job Card
  UPDATE cmms_jobrequest_mst: status=ASSIGNED, approved fields, engineer, linked JC
  INSERT history: APPROVED -> ASSIGNED
  INSERT audit_log: JR_CONVERT
  INSERT audit_log: JC_CREATE
COMMIT
```

Write target	Why it is written
job_request_status_history #1	Shows the approval decision in the timeline.
cmms_jobcard_mst	Creates the executable Job Card.
cmms_jobrequest_mst	Marks parent request assigned and linked to Job Card.
job_request_status_history #2	Shows assignment after approval.
audit_log JR_CONVERT	Records the business decision against the Job Request.
audit_log JC_CREATE	Records the new Job Card entity creation.

ROLLBACK RULE If any statement fails after BEGIN, the service rolls back. The strongest test is an injected failure after the Job Card insert: after rollback, JR status, Job Card row, history rows, and audit rows must all remain unchanged.

Part XI — Reject Workflow Deep Dive

Reject is intentionally separate from Convert. It is terminal for Slice 2, requires a reason, and never creates a Job Card. This prevents accidental work creation for a rejected request and keeps audit trace clean.

Aspect	Locked behavior
Allowed starting state	SUBMITTED only.
Final state	REJECTED.
Reason	Mandatory; stored in JR_REJECTION_REASON and status history.
Job Card creation	None.
Audit action	JR_REJECT.
Cache effect	Invalidate relevant org/personal/list caches.

```
reject(jrNo, reason, actor):
  begin transaction
  lock JR row
  require status == SUBMITTED
  transition(SUBMITTED, reject, actor)
  update JR to REJECTED + rejected_by/on/reason
  append history SUBMITTED -> REJECTED with reason
  write JR_REJECT audit row
  commit
```

BUSINESS MEANING Rejected is terminal for Slice 2. The system does not reopen rejected requests yet. If business later needs reopen, it must be added deliberately with a new transition and audit rule.

Part XII — Job Request Detail Page Deep Dive

The Job Request Detail page turns a list row into an audit-grade object. A manager can understand the equipment, who submitted it, what the complaint is, what accessories were submitted, where the request is in the state machine, and whether a Job Card has already been created.

Component	Purpose
DetailHeader	Back link, JR code, status pill, priority pill, job type, submitted/updated timestamps.
DetailEquipmentCard	Equipment name, make, model, serial number, type, options/description.
DetailSubmissionCard	Submitter identity, designation, email, phone, division, project, subsystem.
DetailComplaintCard	Complaint/symptoms, remarks, accessories table.
DetailTimelineCard	Full chronological status_history with actor, timestamp, and reason.
DetailLinkedJobCardCard	Linked Job Card summary after conversion.
DetailActionBar	Convert and Reject buttons when status and permissions allow.

Detail authorization logic

Role	Detail visibility
Normal User	Own Job Requests only; foreign rows denied by row-level scope.
View-Only	Read-only access where read-all permission exists.
Lab Engineer	Read access to global job-flow context.
Lab In-Charge	Read all, convert, and reject.
Super Admin	Read all, convert, and reject.

UI HONESTY RULE The frontend hides Convert/Reject actions unless the user has the right permissions and the request is SUBMITTED. The backend still enforces permissions and state; UI hiding is convenience, not security.

Part XIII — Conversion Workspace Deep Dive

The Conversion workspace is the batch-processing surface for Lab In-Charge and Super Admin. It organizes submitted requests by operational intent, surfaces filters, and provides in-place Convert/Reject modals so the user does not lose queue context.

Tab	Mapped job_type	Operational meaning
Calibration	CALIBRATION	Requests that should become calibration Job Cards.
Inspection	REPAIR	Repair/inspection style requests requiring investigation.
Master Data Correction	REGISTRATION	Registration and master-data correction requests.

Conversion table behavior

- Rows are fetched from the existing Job Request list endpoint using status=SUBMITTED and job_type filter.
- The table shows enough information to decide: ID, equipment, submitter, priority, status, and action buttons.
- The filter strip supports search, priority, and date range without creating a new backend concept.
- The page polls/refetches to keep tab counts close to current after conversion or rejection.
- Post-convert UX stays on /conversion for batch processing instead of navigating to the new Job Card.

BATCH-PROCESSING DECISION Staying on /conversion after Convert is intentional. LIC users are likely processing many submitted requests. Navigation to a detail page would break throughput.

Part XIV — RBAC, Scope, Audit, and Security

Security in this slice is not a single middleware. It is the combination of route permission gates, row-level scope, state-machine permission checks, service-level transaction ownership, and audit rows written inside the same transaction.

New endpoint map

Method	Path	Permission model	Purpose
GET	/api/v1/job-requests/:id	read-own OR read-all + row-level scope	Full Job Request detail.
GET	/api/v1/job-requests/:id/history	same as detail	Status timeline rows.
POST	/api/v1/job-requests/:id/convert	job_request:approve AND job_request:assign-engineer	Approve+assign+auto-create Job Card.
POST	/api/v1/job-requests/:id/reject	job_request:reject	Reject submitted request with reason.
GET	/api/v1/lookups/engineers	job_request:assign-engineer	Engineer dropdown with active workload count.

Audit vocabulary

Action	Entity	When it is written
JR_CONVERT	job_request	Convert succeeds.
JC_CREATE	job_card	Convert creates the Job Card.
JR_REJECT	job_request	Reject succeeds.

AUDIT DOCTRINE Audit rows are not decoration. They are part of the transaction story. If the state changes, audit must exist. If audit exists, the state change must have committed.

Failure modes blocked

Failure mode	Blocked by
Normal user tries convert	Route authorize + state-machine permission check.
Convert non-SUBMITTED JR	SELECT FOR UPDATE + state-machine illegal transition.
Assign inactive/non-engineer	Engineer lookup/validation/service check.
Workflow mismatch	Frontend bucket + backend validator/service validation.
Partial convert	Single transaction and rollback on error.
Missing timeline	Two status_history inserts inside conversion transaction.

Part XV — Verification Matrix A1 → A14

The transcript defines the acceptance matrix for the slice. The build must not be called sealed until these conditions pass in the local environment and, later, in staging.

ID	Test	Expected
A1	GET detail as Normal · own JR	200 + body
A2	GET detail as Normal · others JR	403 FORBIDDEN
A3	GET detail as LIC · any JR	200 + body
A4	POST convert as Normal	403 FORBIDDEN
A5	POST convert valid body as LIC	JR=ASSIGNED + JC=ASSIGNED + 2 history + 2 audit
A6	POST convert where jr.status != SUBMITTED	409 ILLEGAL_TRANSITION
A7	POST convert inactive/non-engineer	400 BAD_REQUEST
A8	POST convert workflow_type mismatch	400 BAD_REQUEST
A9	POST convert planned_start < received	400 BAD_REQUEST
A10	POST reject reason length < 10	422 VALIDATION_ERROR
A11	POST reject valid as LIC	JR=REJECTED + 1 history + 1 audit + no JC
A12	Forced rollback inside convert txn	JR unchanged, no JC, no history, no audit
A13	/conversion tab badges after mutation	Counts match DB within refresh window
A14	Dashboard Pending Jobs after convert	Pending count decreases by one within refresh window

HIGHEST-VALUE TEST A12 is the strongest test. It proves the transaction is real, not just hopeful. If A12 fails, the slice is not sealed.

Build checks visible in transcript

Check	Result
Migrations 200/201	Applied through the correct phase3 migration runner.
New columns	JM_ASSIGNED_ENGINEER, JM_WORKFLOW_TYPE, JM_REQUIRED_RESOURCES, JM_SPECIAL_INSTRUCTIONS, JM_PARENT_JR_NO verified.
New indexes	idx_jc_engineer_status and idx_jc_parent_jr verified.
Backend module load	Routes/repos/services loaded successfully in Node sanity-load test.
Bootstrap verifier drift	Expected post-phase drift noted: permissions/users totals no longer match original bootstrap-only counts.

Part XVI — Risk Register and Troubleshooting

Risk	Symptom	Likely cause	Fix
Duplicate Job Card number	Convert fails with PK collision	Sequence generator race or wrong lock	Ensure MAX(JM_JobCardNO)+1 is computed inside transaction with lock semantics.
APPROVED missing from JR list	Developer expects JR_MVP_STATUS=APPROVED	Misunderstood transient decision	Use status_history for APPROVED; persisted state is ASSIGNED.
Engineer dropdown empty	Convert modal cannot assign	No active LAB_ENGINEER users or permission gate blocking lookup	Seed engineer users through Admin module and verify role_permissions.
Convert succeeds but dashboard stale	Pending Jobs count not updated immediately	Cache not invalidated or FE polling delay	Check kpiCache invalidation and refresh interval.
Reject creates Job Card	Unexpected JC row after rejection	Service path wrongly shared with convert	Keep reject transaction separate.
Normal sees foreign detail	Data leak	rowLevelScope not passed or repo ignored scope	Enforce service/repo ownership check.

Troubleshooting doctrine

- First inspect the state machine. If the state transition is illegal there, do not patch around it in the service.
- Then inspect the service transaction. If there is any write outside the transaction, the slice is unsafe.
- Then inspect route middleware. Convert must require both approve and assign-engineer permissions.
- Then inspect cache invalidation. UI correctness after mutation depends on invalidating detail, history, conversion list, engineers lookup, and dashboard/list caches.
- Finally inspect frontend schema buckets. Workflow mismatch should be prevented in the UI and rejected by the backend.

Part XVII — Handover Notes and Next Slice

HANDOVER STATUS Phase 7 Slice 2 is the workflow bridge: submitted Job Requests can now become assigned Job Cards, and invalid requests can be rejected with audit-grade traceability.

What future engineers must preserve

- Keep the Convert transaction atomic. Do not split approve and assign into separate commits unless a future phase introduces an explicit partial-approval state and migration.
- Keep APPROVED transient unless the database enum is deliberately changed through a reviewed migration.
- Keep rejected requests terminal until a formal reopen flow is designed.
- Keep the engineer identity as employee_id unless the entire schema moves to numeric user_id references.
- Keep frontend action hiding as a UX optimization, never as the only security boundary.
- Keep status_history and audit_log writes coupled to state mutation.

Recommended next work

Next unit	Scope
Phase 7 Slice 3 or Phase 9 Job Card Execution	Job Card detail, Start Work, observations/readings, complete, verify/close, reopen.
Reports/Exports	Calibration due, pending jobs, engineer workload, PDF/Excel exports.
Equipment detail integration	Show linked Job Cards and calibration history on equipment detail.
More engineer seed data	Add multiple Lab Engineer users so workload balancing is meaningful in demonstrations.

Appendix A — File Inventory

Area	Files / components
Database migrations	200__phase7s2_jc_columns.sql, 201__phase7s2_indexes.sql
Discovery docs	SCHEMA_PHASE7_SLICE2.md, introspection output file
Backend Job Requests	stateMachine, repo, service, controller, routes, validators
Backend Job Cards	repo extensions for sequence and insert
Backend Lookups	engineers lookup in repo/controller/routes
Frontend lib	jobRequestSchemas, jobRequests API, lookups API, detail/history/engineer/conversion hooks
Frontend routes	App.jsx entries for /job-requests/:id and /conversion
Frontend Detail	JobRequestDetail and 7 sub-components
Frontend Conversion	Conversion page, tabs, table, Convert modal, Reject modal

Appendix B — Pseudo-code and Route Reference

```
POST /api/v1/job-requests/:id/convert
  requires: authenticate + approve + assign-engineer
  body: engineer_employee_id, workflow_type, equipment_received_date,
        planned_start_date, target_end_date, resources?, instructions?
  returns: { job_request, job_card }

POST /api/v1/job-requests/:id/reject
  requires: authenticate + job_request:reject
  body: { reason }
  returns: { job_request }

GET /api/v1/lookups/engineers
  requires: authenticate + job_request:assign-engineer
  returns: active LAB_ENGINEER users sorted by active_card_count
```

Appendix C — Glossary

Term	Meaning
Convert	The combined approve + assign + auto-create Job Card action.
Transient APPROVED	Logical approval step captured in history but not persisted as JR row status.
Status history	Append-only transition log for Job Request lifecycle.
Audit log	Operational accountability record for business actions.
Row-level scope	Backend-enforced visibility rule that restricts Normal users to their own requests.
Workflow type	Operational flavor of the Job Card, scoped by job_type.

Part XVIII — Logic Development Playbook

HOW TO THINK This part is intentionally written as a senior-engineering playbook. It explains how the database, backend, and frontend logic should be reasoned about before adding or changing code in this slice.

18.1 Database logic development — from requirement to column

Start with the business verb, not the table. The business verb is Convert: a submitted request becomes assigned work. Then ask which facts must become durable after that verb commits. In this slice the durable facts are: who approved, who is assigned, what Job Card was created, which workflow applies, what the target timeline is, and why a request was rejected if it was not accepted.

Thinking question	Senior answer for Slice 2
Can the existing Job Request row store approval?	Yes. JR_APPROVED_BY and JR_APPROVED_ON already exist.
Can it store rejection?	Yes. JR_REJECTED_BY, JR_REJECTED_ON, and JR_REJECTION_REASON already exist.
Can it store the linked Job Card?	Yes. JR_SECTIONJOB_NO already points to JM_SectionJobNo.
Can the Job Card store assignment and workflow?	Not completely; add NULLable fields on Job Card only.
Can APPROVED be persisted?	No. The enum does not contain APPROVED; store it in history only.

DATABASE THINKING RULE Do not add a column because the UI has a label. Add a column only when a business fact must survive page reload, restart, audit, and future reporting.

18.2 Backend logic development — from endpoint to transaction

The backend starts from the user action. If the action is Convert, the endpoint is not “update Job Request status”; it is “perform a business transaction”. That distinction decides the service design. The controller should not know the steps. The route should not know SQL. The repository should not know why the transition matters. The service composes all of it.

Layer	Question to ask	Correct Slice 2 answer
Route	Who is allowed to ask?	authenticate + approve + assign-engineer for Convert.
Validator	What shape of input is acceptable?	Engineer, workflow, and ordered dates are mandatory.
Service	What must happen together?	Lock JR, state checks, insert JC, update JR, history, audit, cache invalidation.
Repository	What exact SQL writes are needed?	Insert/update/select functions using existing real columns.
State machine	Is this transition legal?	SUBMITTED→APPROVED→ASSIGNED or SUBMITTED→REJECTED only.

```
A good service method reads like a business story:
  1. lock the thing being decided
  2. prove the actor may decide it
  3. create/update all durable facts
  4. write the audit trail
  5. commit once
  6. invalidate caches after commit
```

18.3 Frontend logic development — from user intention to state refresh

The frontend starts from the user mental model. Lab In-Charge users do not want to lose the queue after every decision. They want to process rows quickly. Therefore the conversion modal closes after success, the page stays on the same tab, and caches refetch. The UI is built to preserve operating context.

UI problem	Implementation answer
How does LIC choose engineer?	Dropdown sourced from /lookups/engineers with active_card_count.
How to prevent wrong workflow?	Workflow dropdown is bucketed by job_type.
How to prevent bad dates?	HTML date constraints + zod superRefine + backend validation.
How to keep list current?	Invalidate conversion list and poll/refetch after mutation.
How to keep detail current?	Invalidate detail and history caches after Convert/Reject.

FRONTEND THINKING RULE A hidden button is not security. It is only a cleaner interface. Backend authorization and state-machine checks remain the real guard.

Part XIX — Backend Module Deep Review

19.1 jobRequests.stateMachine.js

The state machine is pure logic. It does not import database connections. It does not know controllers. It accepts a current state, an action, and an actor, then returns the new state or throws. This keeps the lifecycle explainable and testable.

Rule	Why it exists
DRAFT submit owner-only	Prevents another user from submitting someone else’s draft.
SUBMITTED approve	Allows LIC/SA to record managerial approval.
APPROVED assign	Separates assignment permission from approval permission.
SUBMITTED reject	Allows terminal refusal with reason.
No transitions from REJECTED	Prevents accidental reopen without designed business flow.

19.2 jobRequests.service.js

The service is the senior-engineering center of the slice. It holds the transaction boundary and is therefore where correctness is decided. Every function added here should be read through four lenses: authorization, state, write set, and audit.

Service function	Logic responsibility
getDetail	Read detail with row-level visibility rules and linked objects.
getHistory	Read ordered status_history rows for the timeline.
convertToJobCard	Atomically approve, assign, create Job Card, write history/audit, invalidate caches.
reject	Atomically mark rejected, write reason, history, audit, invalidate caches.

SERVICE SMELL If a future function in this module writes to JR_MVP_STATUS and does not also write status_history and audit intent, it is probably wrong.

19.3 jobRequests.repo.js and jobCards.repo.js

The repositories are intentionally boring. Boring repositories are good repositories. They know real column names and return predictable canonical shapes. They do not make business decisions.

Repository rule	Slice 2 example
SQL is contained	Only repo files know JR_JOBREQUESTNO, JM_SectionJobNo, JR_SECTIONJOB_NO.
Services receive canonical payloads	Detail card components do not know legacy column names.
Transactions pass connection object	Convert uses the same connection for lock, insert, update, history, audit.
Generators are server-side	nextSectionJobNo is not a frontend responsibility.

19.4 lookups module extension

The engineers endpoint looks small but carries product significance. It turns assignment from blind selection into workload-aware decision-making. It also keeps the Convert modal from hardcoding users or roles.

Returned field	Use in UI
employee_id	Stored into JR_ASSIGNED_ENGINEER and JM_ASSIGNED_ENGINEER.
full_name	Shown in dropdown for human selection.
division_code	Context for organizational assignment.
active_card_count	Workload signal sorted ascending.

LOOKUP SECURITY The engineers lookup is gated by job_request:assign-engineer. A user who cannot assign should not be able to query the assignment surface.

Part XX — Frontend Component Deep Review

20.1 JobRequestDetail page composition

The detail page is not a form. It is an operational dossier. It lets the reviewer see the entire request before deciding. The cards are intentionally separated so a future engineer can upgrade one block without destabilizing the rest of the page.

Component	Why it is separated
DetailHeader	High-level status and identity can evolve independently.
DetailEquipmentCard	Equipment fields may later link to Equipment detail.
DetailSubmissionCard	Submitter snapshot logic is sensitive and should remain isolated.
DetailComplaintCard	Complaint and accessories are the core operational evidence.
DetailTimelineCard	Timeline can later merge audit rows without touching other cards.
DetailLinkedJobCardCard	Only appears after conversion; optional rendering stays clean.
DetailActionBar	Permission/state logic for actions is centralized.

20.2 ConvertToJobCardModal

The Convert modal collects only the facts needed to create a usable Job Card. It does not ask the manager to re-enter equipment, submitter, or complaint data because those facts already exist on the Job Request and are copied server-side.

Modal section	Senior design reason
Assignment & Workflow	This is the managerial decision: who should do what kind of work.
Timeline	This converts an abstract request into scheduled work.
Instructions	This reduces information loss between LIC and engineer.
Save as Draft disabled	Visible roadmap signal without shipping partial incomplete persistence.

MODAL CLOSE RULE Overlay click is intentionally not the main close path for dirty forms. Losing a half-filled conversion could cause operational mistakes.

20.3 RejectModal

The Reject modal is deliberately smaller and stricter. Rejection is terminal in Slice 2, so the UI must make the consequence clear and require a reason that is long enough to be useful in audits.

Field	Rule
reason	Required, minimum length, capped to fit database/audit notes safely.
submit	Calls rejectJobRequest and invalidates caches on success.
close	Cheap to close because only one field exists.

Part XXI — Transaction Narratives

21.1 Happy path: Convert

A Lab In-Charge opens /conversion, sees a submitted calibration request, opens Convert modal, selects engineer TE00225, chooses CALIBRATION_STANDARD, sets dates, adds instructions, and clicks Create Job Card. The backend locks the JR row, validates it is still SUBMITTED, records the approval step, creates the Job Card, links it, commits, and returns the new identifiers. The UI closes the modal and refreshes the queue.

Time	System fact after step
Before click	JR status is SUBMITTED and no linked Job Card exists.
During transaction	JR row is locked; no competing conversion can safely alter it.
After Job Card insert	New JM_SectionJobNo exists only inside transaction until commit.
After JR update	JR points to the new Job Card and status is ASSIGNED.
After commit	Timeline and audit show the conversion and creation.

21.2 Happy path: Reject

A Lab In-Charge opens a submitted request, determines it is invalid, clicks Reject, writes a reason, and submits. The backend locks the row, validates status and permission, writes rejected fields, appends history, writes audit, commits, and returns the updated state. No Job Card is created.

Fact	Expected after reject
JR_MVP_STATUS	REJECTED
JR_REJECTED_BY	actor employee_id
JR_REJECTED_ON	current timestamp
JR_REJECTION_REASON	submitted reason
Job Card row	none created
Audit	one JR_REJECT row

21.3 Conflict path: stale conversion page

A manager may open a conversion modal while another authorized user converts the same Job Request first. When the first manager submits, the backend row lock/state check detects that the row is no longer SUBMITTED and returns an illegal-transition conflict instead of creating a second Job Card.

CONCURRENCY PROTECTION The UI can be stale; the database transaction cannot be. Always trust the backend row lock and state check over the visible page state.

Part XXII — Reviewer Checklist

Use this checklist before accepting any future edit to Phase 7 Slice 2.

Checklist item	Pass condition
Migrations additive-only	No DROP, no RENAME, no NOT NULL on legacy data.
State changes through state machine	No raw JR_MVP_STATUS mutation outside service/state machine path.
Convert transaction atomic	All writes share one connection and one commit.
Reject separate from convert	Reject never calls Job Card creation path.
Routes gated	Convert requires both approve and assign-engineer; reject requires reject.
Row-level detail visibility	Normal users cannot view others rows.
Timeline complete	Convert creates two history rows; reject creates one.
Audit complete	Convert creates JR_CONVERT + JC_CREATE; reject creates JR_REJECT.
Caches invalidated	Detail, history, conversion list, engineers lookup, and dashboards/lists refresh.

Part XXIII — Demo Script

This demo script is designed for showing the slice to a mentor, stakeholder, or future evaluator without diving into code.

Step	Demo action	Expected visible result
1	Login as Lab In-Charge or Super Admin.	Sidebar exposes Job Requests and Conversion workspace.
2	Open /conversion.	Tabs show submitted Calibration, Inspection, and Master Data Correction queues.
3	Open a submitted request detail.	Full equipment, submitter, complaint, accessory, and timeline view appears.
4	Click Convert to Job Card.	Modal opens with engineer/workflow/timeline/instructions sections.
5	Select engineer and valid workflow, submit.	Modal closes; queue refreshes; request is no longer pending.
6	Reopen detail for converted request.	Linked Job Card card appears and timeline shows approval/assignment.
7	Reject another submitted request.	Reason is recorded; no Job Card appears.

Part XXIV — Final Engineering Memory

REMEMBER THIS The phrase Auto Job Card does not mean a simple INSERT. It means a controlled business conversion from submitted request to assigned work, implemented as a transaction with state-machine authorization, status history, audit rows, cache refresh, and a human-grade UI.

Final Seal

SEALED CONCLUSION Phase 7 Slice 2 is now documented as the bridge between submitted requests and executable work. The central engineering truth is atomic workflow conversion: one decision, one transaction, one traceable timeline, one assigned Job Card.

End of document.